

Today 11:12 AM

**main(1).py**

Code

**game_engine(1).py**

Code

**target(1).py**

Code

**README(1).md**

Document

I'm working on my Software Engineering Lab 4 assignment. The project assigned to me is **Target Aim Trainer**, a small Pygame game.

I've uploaded the current project files and README. Please first understand the existing code and the requirements from the README before making any changes.

I have already run the original project and recorded the required **10-second video before making changes**.

For now, I only want to work on **Task 1**. Please don't implement the other tasks yet.

Task 1 – Fix Collision Detection

There is a bug in the current game:

- The target starts as a large circle and gradually shrinks while it stays on the screen.
- The circle that is actually drawn becomes smaller over time.
- However, the clickable/collision area is still based on the original target size.
- Because of this, when the target has become small, I can click outside the visible circle and the game may still count it as a hit.

I want the clickable area to always match the **current visible size of the target**.

Please:

1. Look at the uploaded code and identify exactly why this is happening.
2. Explain the problem in simple terms.
3. Suggest the smallest and cleanest code change needed to fix it.
4. Give me the updated code for the file(s) that actually need to be changed.
5. Do **not** implement Game Over, Replay/Difficulty, or Sound yet. Those are separate tasks that I will do later.
6. Don't unnecessarily rewrite or restructure the project.

I'm a student doing this as a Vibe Coding lab, so please keep the solution simple and understandable rather than making it overly complicated.

Also tell me how I can test the fix manually after making the change.

I reviewed the uploaded `README.md` , `main.py` , `game_engine.py` , and `target.py` . The README confirms that **Task 1 is specifically about making the clickable area match the shrinking target**, while Tasks 2–4 are separate and should not be implemented yet.

 README

1. Why the bug happens

The problem is entirely in `game/target.py` .

Your target has two different radius calculations:

- `visual_radius()` → calculates the **current radius** of the target as it shrinks.
- `contains_point()` → checks whether the mouse click is inside the target.

The drawing code correctly uses the changing radius:

 Python

```
r = int(self.target.visual_radius())
pygame.draw.circle(screen, RED, (self.target.x, self.target.y), r)
```

So the visible circle gets smaller over time. `</>` game_engine

But the collision code currently does this:

`</>` Python

```
return math.hypot(self.x - x, self.y - y) <= self.base_radius
```

That means it **always** uses `base_radius`, which starts at `40`, even after the target has shrunk to `12`. The existing comments in the file actually identify this exact issue. `</>` target

In simple terms

Imagine the target starts like this:

`</>` Plain text

```
00000
000000000
00000000000
000000000
00000
```

Initially, a 40-pixel collision radius is correct.

But later it becomes:

`</>` Plain text

```
000
00000
000
```

The **drawing** says: "I'm small."

But the **collision detection** is still saying: "I'm 40 pixels wide."

So you can click beside the visible target and it may still count as a hit.

2. Smallest clean fix

You only need to change **one line** in:

</> Plain text

game/target.py

Change:

</> Python

```
return math.hypot(self.x - x, self.y - y) <= self.base_radius
```

to:

</> Python

```
return math.hypot(self.x - x, self.y - y) <= self.visual_radius()
```

That's it.

This is the cleanest solution because the game already has a `visual_radius()` method specifically calculating the target's current size.

⌕ target

There is **no need** to modify `main.py` or `game_engine.py`.

The game engine already calls:

⌕ Python

```
self.target.contains_point(x, y)
```

when the player clicks. ⌕ game_engine

So fixing `contains_point()` automatically fixes the collision behavior.

3. Updated target.py

Here is the complete file with only the necessary change.

Updated target.py



```
import math
```

```
class Target:
```

```
    def __init__(self, x, y, base_radius=40, min_radius=12, lifespan_frames=90):
```

```
        self.x = x
```

```
        self.y = y
```

```
self.base_radius = base_radius
self.min_radius = min_radius
self.lifespan_frames = lifespan_frames
self.age = 0

def update(self):
    self.age += 1

def expired(self):
    return self.age >= self.lifespan_frames

def visual_radius(self):
    # The target shrinks as it ages, giving the player less time
    # to react the longer it's been on screen.
    t = min(1.0, self.age / self.lifespan_frames)
    return self.base_radius - (self.base_radius - self.min_radius) * t

def contains_point(self, x, y):
    return math.hypot(self.x - x, self.y - y) <= self.visual_radius()
```

Files you need to change

File	Change?
main.py	 No
game/game_engine.py	 No

File	Change?
game/target.py	☒ Yes — one line
README.md	☐ No

This keeps your original project structure and avoids unnecessary rewriting.

4. How to test Task 1 manually

After saving `target.py`, run:

 Bash

```
python main.py
```

Test 1 — Initial target

When the target first appears, it is large.

Click **inside the circle** → it should count as a hit.

Click **outside the circle** → it should count as a miss.

Test 2 — Wait for the target to shrink

Don't click immediately.

Wait until the target becomes noticeably smaller.

Then deliberately click in the **area where the target used to be, but outside its current visible circle**.

For example:

</> Plain text

```
      Old boundary
    ,-----,
   /         \
  |    000    |
  |   00000   |
  |    000    |
   \         /
    '-----'

      ↑
Click here
(outside visible target)
```

That click should now be a **miss**.

Test 3 — Small target

Wait almost until the target reaches its smallest size.

Click directly on the visible small circle → **hit**.

Click just outside the visible circle → **miss**.

That demonstrates that the collision area is now following the visual size.

5. What you should show for your Task 1 evidence

Since you already recorded the required **before-change 10-second video**, don't record another "before" video.

For the **after-change video**, try to show:

1. Run the modified game.
2. Let a target shrink significantly.
3. Click outside the visible small circle but inside its old/large area → **miss**.
4. Click directly on the small visible circle → **hit**.
5. Optionally repeat once to make the behavior obvious.

The README says the submission requires a 10-second video after the changes showing the bug fixed and the new features working, plus the LLM chat page/history.  [README](#)

For **this stage**, however, we should only implement Task 1. **Do not add Game Over, Replay/Difficulty, or Sound yet.**

The actual fix in one sentence

Use `visual_radius()` instead of `base_radius` for hit detection, because `visual_radius()` represents the target's current on-screen size.

Today 7:01 PM

At the moment, when the round timer reaches zero, the game sets the game-over state, but the final result is printed to the terminal instead of being properly shown inside the Pygame window.

I want the game to behave more like a complete game:

- When the round timer reaches zero, normal gameplay should stop.
- A proper **Game Over screen** should appear inside the Pygame window.
- It should clearly show the final score.
- It should clearly show the final accuracy.
- It should give the player a simple instruction telling them what input to press to continue.

- The program should wait for that input gracefully instead of just printing the result to the console.

Please keep the existing game structure and make the smallest reasonable changes needed.

Only implement Task 2. Do not implement replay, difficulty selection, or sound effects yet — those are Tasks 3 and 4 and I'll handle them separately.

Please don't unnecessarily rewrite the whole project.

Before giving me the code, briefly explain:

1. How the current game handles the timer reaching zero.
2. Why the current implementation doesn't provide a proper game-over screen.
3. What files you think actually need to be changed.
4. How your proposed solution will work.

Then give me the complete updated code for the file(s) that need to be changed.

Finally, give me a simple step-by-step way to test Task 2 manually.

Worked for 9s

I checked the current `game_engine.py` , `main.py` , and the Task 2 requirement in the README. The README specifically asks for a screen showing the final score and accuracy after the timer reaches zero, followed by graceful input handling rather than console output.

 README

1. How the current timer handles zero

In `GameEngine.update()` , the timer is reduced every frame:

 Python

```
self.time_left_frames -= 1
```

When it reaches zero:

Python

```
if self.time_left_frames <= 0:
    self.game_over = True
    return
```

So the existing code **does correctly detect the end of the round** and sets `game_over = True`. `game_engine`

Also, because `update()` immediately returns when `game_over` is true, normal target updates stop.

2. Why there isn't a proper Game Over screen

The current `render()` method still draws the target and HUD. At the bottom it only does:

Python

```
print(f"Time's up! Final score: ...")
```

That sends the result to the **terminal**, not the Pygame window. `game_engine`

There is also another small issue: `handle_event()` immediately returns when `game_over` is true:

Python

```
if self.game_over:
    return
```

Therefore, once the game is over, there is currently no way for a keyboard event to be handled by the game-over state. `</> game_engine`

3. Files that need to change

I recommend changing **only two files**:

- `game/game_engine.py` — add the Game Over display and handle the Enter key.
- `main.py` — allow the engine to tell the main loop that the player has chosen to exit.

`target.py` does not need to change for Task 2.

This also avoids accidentally implementing Task 3. We will **not** add replay or difficulty selection.

How the solution will work

When the timer reaches zero:

1. `game_over` becomes `True`.
2. Normal target updates stop.
3. The target is no longer drawn.
4. A Game Over screen is drawn inside the Pygame window.
5. It displays:
 - `GAME OVER`
 - final score
 - final accuracy
 - `Press ENTER to exit`
6. The game waits for Enter.
7. Enter tells the main loop to close the game cleanly.

Using **Enter to exit** is intentional here: replay/difficulty selection belongs to Task 3 according to the README, so we shouldn't introduce it into Task 2. [📖 README](#)

4. Updated game/game_engine.py

The changes are kept deliberately small.

Updated game_engine.py



```
import pygame
import random
from .target import Target

# Game Engine

WHITE = (255, 255, 255)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.margin = 60
        self.hud_height = 60
        self.target = self._spawn_target()

        self.round_seconds = 30
        self.time_left_frames = self.round_seconds * 60

        self.hits = 0
```

```
self.misses = 0
self.score = 0
self.font = pygame.font.SysFont("Arial", 26)
self.game_over = False
self.should_exit = False

def _spawn_target(self):
    x = random.randint(self.margin, self.width - self.margin)
    y = random.randint(self.margin + self.hud_height, self.height - self.margin)
    return Target(x, y)

def handle_event(self, event):
    if self.game_over:
        if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
            self.should_exit = True
        return

    if event.type == pygame.MOUSEBUTTONDOWN:
        self._handle_click(event.pos)

def _handle_click(self, pos):
    x, y = pos
    if self.target.contains_point(x, y):
        self.hits += 1
        self.score += 1
        self.target = self._spawn_target()
    else:
        self.misses += 1
```

```
def handle_input(self):
    # Reserved for continuously-held-key input; this game is
    # entirely mouse-driven, so there's nothing to poll here.
    pass

def update(self):
    if self.game_over:
        return

    self.time_left_frames -= 1
    if self.time_left_frames <= 0:
        self.game_over = True
        return

    self.target.update()
    if self.target.expired():
        self.misses += 1
        self.target = self._spawn_target()

def accuracy(self):
    total = self.hits + self.misses
    if total == 0:
        return 0.0
    return round(100 * self.hits / total, 1)

def render(self, screen):
    if self.game_over:
```

```
game_over_text = self.font.render("GAME OVER", True, WHITE)
score_text = self.font.render(
    f"Final Score: {self.score}", True, WHITE
)
accuracy_text = self.font.render(
    f"Final Accuracy: {self.accuracy()}%", True, WHITE
)
instruction_text = self.font.render(
    "Press ENTER to exit", True, WHITE
)

screen.blit(
    game_over_text,
    (
        self.width // 2 - game_over_text.get_width() // 2,
        120
    )
)
screen.blit(
    score_text,
    (
        self.width // 2 - score_text.get_width() // 2,
        190
    )
)
screen.blit(
    accuracy_text,
    (
```

```
        self.width // 2 - accuracy_text.get_width() // 2,
        240
    )
)
screen.blit(
    instruction_text,
    (
        self.width // 2 - instruction_text.get_width() // 2,
        320
    )
)
return

r = int(self.target.visual_radius())
pygame.draw.circle(screen, RED, (self.target.x, self.target.y), r)
pygame.draw.circle(screen, WHITE, (self.target.x, self.target.y), r, 2)

score_text = self.font.render(f"Score: {self.score}", True, WHITE)
screen.blit(score_text, (10, 10))

seconds_left = max(0, self.time_left_frames // 60)
timer_text = self.font.render(f"Time: {seconds_left}s", True, WHITE)
screen.blit(timer_text, (self.width - 140, 10))

acc_text = self.font.render(f"Accuracy: {self.accuracy()}%", True, WHITE)
screen.blit(acc_text, (self.width // 2 - 90, 10))
```

What changed?

There are only three important changes:

1. Added an exit flag:

</> Python

```
self.should_exit = False
```

2. Game-over input is now handled:

</> Python

```
if self.game_over:
    if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
        self.should_exit = True
    return
```

3. `render()` now displays a Game Over screen instead of printing to the terminal.

The normal target/HUD rendering is skipped when `game_over` is true.

5. Updated `main.py`

We only need a tiny change here so the main loop can see when Enter was pressed.

Updated main.py



```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 700, 500
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Target Aim Trainer - Pygame Version")

# Colors
DARK_GRAY = (35, 35, 40)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True
    while running:
        SCREEN.fill(DARK_GRAY)

        for event in pygame.event.get():
            if event.type == pygame.QUIT:
```

```
        running = False

    engine.handle_event(event)

    if engine.should_exit:
        running = False

    engine.handle_input()
    engine.update()
    engine.render(SCREEN)

    pygame.display.flip()
    clock.tick(FPS)

pygame.quit()

if __name__ == "__main__":
    main()
```

The rest of the project stays untouched.

6. How to test Task 2

Because the round currently lasts **30 seconds**, the easiest test is:

Step 1

Run:

</> Bash

```
python main.py
```

Step 2

Play normally for about 30 seconds.

You should see:

- target spawning
- target shrinking
- score changing
- accuracy changing
- countdown decreasing

Step 3

Don't worry about clicking after the timer reaches zero.

When the timer reaches:

</> Plain text

```
Time: 0s
```

the normal gameplay should stop.

Step 4

Instead of the terminal showing something like:

</> Plain text

Time's up! Final score: 5 Accuracy: 71.4%

you should now see a screen inside the Pygame window similar to:

</> Plain text

GAME OVER

Final Score: 5

Final Accuracy: 71.4%

Press ENTER to exit

Step 5

Try clicking the mouse after Game Over.

Nothing should happen.

This confirms normal gameplay has stopped.

Step 6

Press **Enter**.

The Pygame window should close cleanly and return you to the terminal.

Task 2 result

So the flow is now:

</> Plain text

```
Normal Gameplay
↓
Timer reaches 0
↓
game_over = True
↓
Normal target updates stop
↓
Game Over screen appears
↓
Final Score + Accuracy shown
↓
"Press ENTER to exit"
↓
Player presses Enter
↓
Game closes
```

This implements **Task 2 only**. There is deliberately **no replay**, **no difficulty selection**, and **no sound**, leaving those for Tasks 3 and 4 as specified in the README. [📖 README](#)

After the Game Over screen, I want the player to be able to choose what to do next:

- Easy
- Medium
- Hard
- Exit

Each difficulty should make the target behave differently by changing its difficulty-related properties such as target lifespan and/or target size.

When the player chooses a difficulty:

- Start a completely new 30-second round.
- Reset the score.
- Reset hits and misses.
- Reset accuracy.
- Spawn a new target.
- Apply the selected difficulty.

After another round ends, the player should again be able to choose a difficulty or exit.

Please keep the existing Task 1 collision fix and Task 2 Game Over functionality intact.

Do not implement Task 4 (sound) yet.

Please don't unnecessarily rewrite the whole project or introduce complicated architecture. Keep it simple and suitable for a Software Engineering lab.

Before giving me the code, please explain:

1. How you plan to represent the three difficulty levels.
2. Which target properties will change for Easy, Medium, and Hard.
3. How the game will transition from Game Over → difficulty selection → new game.
4. Which files actually need to be modified.

Then provide the complete updated code for the files that need changes.

Finally, explain how I can manually test each difficulty and verify that replay correctly resets the game state.

Please implement **Task 3 only**. Don't add sound or any other features yet

Yes. For Task 3, we can build directly on the Task 1 and Task 2 solution without changing the overall architecture.

The important point is that **Task 3 should turn the existing Game Over state into a difficulty-selection state**, rather than creating a separate complicated game-management system.

1. How I will represent the difficulties

I'll keep the three difficulties as simple configuration values inside `GameEngine` :

Difficulty	Target size	Minimum size	Lifespan	Behavior
Easy	50 px	18 px	120 frames	Larger + stays longer
Medium	40 px	12 px	90 frames	Current/default behavior
Hard	30 px	8 px	60 frames	Smaller + disappears faster

At 60 FPS, that means approximately:

- Easy → 2 seconds
- Medium → 1.5 seconds
- Hard → 1 second

The existing target already supports `base_radius` , `min_radius` , and `lifespan_frames` , so we don't need to redesign `Target` .

```
</> target
```

And the Task 1 collision fix remains intact because collision detection will continue using the target's current `visual_radius()` .

2. Game Over → Difficulty Selection → New Game

The flow will become:

</> Plain text

```
30-second round
↓
Timer reaches 0
↓
GAME OVER
↓
Choose:
  1. Easy
  2. Medium
  3. Hard
  4. Exit
↓
Player selects difficulty
↓
Reset:
  Score = 0
  Hits = 0
  Misses = 0
  Timer = 30 seconds
  New target
  Selected difficulty
↓
New 30-second round
↓
Game Over again
↓
Difficulty selection again
```

This means the player can repeatedly play new rounds.

For simplicity, I'll use **keyboard keys 1 , 2 , 3 , and 4** . This avoids adding mouse-button menu logic and keeps the change small.

3. Files that need modification

Only:

⟨/⟩ Plain text

```
game/game_engine.py  
main.py
```

target.py

No change needed.

Your Task 1 collision fix stays exactly as it is:

⟨/⟩ Python

```
return math.hypot(self.x - x, self.y - y) <= self.visual_radius()
```

The difficulty settings will simply change the target's existing size/lifespan parameters.

4. Updated game/game_engine.py

Here is the complete updated file.

Updated game_engine.py — Task 3



```
import pygame  
import random  
from .target import Target
```

```
# Game Engine
```

```
WHITE = (255, 255, 255)
```

```
RED = (220, 60, 60)
```

```
class GameEngine:
```

```
    def __init__(self, width, height):
```

```
        self.width = width
```

```
        self.height = height
```

```
        self.margin = 60
```

```
        self.hud_height = 60
```

```
        self.round_seconds = 30
```

```
        self.difficulties = {
```

```
            "Easy": {
```

```
                "base_radius": 50,
```

```
                "min_radius": 18,
```

```
                "lifespan_frames": 120
```

```
            },
```

```
            "Medium": {
```

```
                "base_radius": 40,
```

```
                "min_radius": 12,
```

```
                "lifespan_frames": 90
```

```
            },
```

```
            "Hard": {
```

```
        "base_radius": 30,
        "min_radius": 8,
        "lifespan_frames": 60
    }
}

self.difficulty = "Medium"

self.hits = 0
self.misses = 0
self.score = 0
self.font = pygame.font.SysFont("Arial", 26)

self.game_over = False
self.should_exit = False

self._start_new_game()

def _spawn_target(self):
    x = random.randint(self.margin, self.width - self.margin)
    y = random.randint(
        self.margin + self.hud_height,
        self.height - self.margin
    )

    settings = self.difficulties[self.difficulty]

    return Target(
```

```
        x,
        y,
        settings["base_radius"],
        settings["min_radius"],
        settings["lifespan_frames"]
    )

def _start_new_game(self):
    self.hits = 0
    self.misses = 0
    self.score = 0
    self.time_left_frames = self.round_seconds * 60
    self.game_over = False
    self.target = self._spawn_target()

def handle_event(self, event):
    if self.game_over:
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_1:
                self.difficulty = "Easy"
                self._start_new_game()

            elif event.key == pygame.K_2:
                self.difficulty = "Medium"
                self._start_new_game()

            elif event.key == pygame.K_3:
                self.difficulty = "Hard"
```

```
        self._start_new_game()

    elif event.key == pygame.K_4:
        self.should_exit = True

    return

if event.type == pygame.MOUSEBUTTONDOWN:
    self._handle_click(event.pos)

def _handle_click(self, pos):
    x, y = pos

    if self.target.contains_point(x, y):
        self.hits += 1
        self.score += 1
        self.target = self._spawn_target()
    else:
        self.misses += 1

def handle_input(self):
    # Reserved for continuously-held-key input; this game is
    # entirely mouse-driven, so there's nothing to poll here.
    pass

def update(self):
    if self.game_over:
        return
```

```
self.time_left_frames -= 1

if self.time_left_frames <= 0:
    self.game_over = True
    return

self.target.update()

if self.target.expired():
    self.misses += 1
    self.target = self._spawn_target()

def accuracy(self):
    total = self.hits + self.misses

    if total == 0:
        return 0.0

    return round(100 * self.hits / total, 1)

def render(self, screen):
    if self.game_over:
        game_over_text = self.font.render(
            "GAME OVER",
            True,
            WHITE
        )
```

```
score_text = self.font.render(  
    f"Final Score: {self.score}",  
    True,  
    WHITE  
)  
  
accuracy_text = self.font.render(  
    f"Final Accuracy: {self.accuracy()}%",  
    True,  
    WHITE  
)  
  
difficulty_text = self.font.render(  
    f"Difficulty: {self.difficulty}",  
    True,  
    WHITE  
)  
  
easy_text = self.font.render(  
    "1 - Easy",  
    True,  
    WHITE  
)  
  
medium_text = self.font.render(  
    "2 - Medium",  
    True,
```

```
        WHITE
    )

    hard_text = self.font.render(
        "3 - Hard",
        True,
        WHITE
    )

    exit_text = self.font.render(
        "4 - Exit",
        True,
        WHITE
    )

    screen.blit(
        game_over_text,
        (
            self.width // 2 - game_over_text.get_width() // 2,
            60
        )
    )

    screen.blit(
        score_text,
        (
            self.width // 2 - score_text.get_width() // 2,
            110
        )
    )
```

```
    )  
)  
  
screen.blit(  
    accuracy_text,  
    (  
        self.width // 2 - accuracy_text.get_width() // 2,  
        150  
    )  
)  
  
screen.blit(  
    difficulty_text,  
    (  
        self.width // 2 - difficulty_text.get_width() // 2,  
        190  
    )  
)  
  
screen.blit(  
    easy_text,  
    (  
        self.width // 2 - easy_text.get_width() // 2,  
        250  
    )  
)  
  
screen.blit(  

```

```
        medium_text,  
        (  
            self.width // 2 - medium_text.get_width() // 2,  
            290  
        )  
    )  
  
    screen.blit(  
        hard_text,  
        (  
            self.width // 2 - hard_text.get_width() // 2,  
            330  
        )  
    )  
  
    screen.blit(  
        exit_text,  
        (  
            self.width // 2 - exit_text.get_width() // 2,  
            370  
        )  
    )  
  
    return  
  
    r = int(self.target.visual_radius())  
  
    pygame.draw.circle(
```

```
        screen,
        RED,
        (self.target.x, self.target.y),
        r
    )

    pygame.draw.circle(
        screen,
        WHITE,
        (self.target.x, self.target.y),
        r,
        2
    )

    score_text = self.font.render(
        f"Score: {self.score}",
        True,
        WHITE
    )

    screen.blit(score_text, (10, 10))

    seconds_left = max(
        0,
        self.time_left_frames // 60
    )

    timer_text = self.font.render(
```

```
        f"Time: {seconds_left}s",
        True,
        WHITE
    )

    screen.blit(
        timer_text,
        (self.width - 140, 10)
    )

    acc_text = self.font.render(
        f"Accuracy: {self.accuracy()}%",
        True,
        WHITE
    )

    screen.blit(
        acc_text,
        (self.width // 2 - 90, 10)
    )
```

5. Updated main.py

Only one small change is required compared with the Task 2 version: the main loop continues running when a difficulty is selected, but closes when `should_exit` becomes `True`.

Updated main.py — Task 3



```
import pygame
from game.game_engine import GameEngine

# Initialize pygame/Start application
pygame.init()

# Screen dimensions
WIDTH, HEIGHT = 700, 500
SCREEN = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("Target Aim Trainer - Pygame Version")

# Colors
DARK_GRAY = (35, 35, 40)

# Clock
clock = pygame.time.Clock()
FPS = 60

# Game loop
engine = GameEngine(WIDTH, HEIGHT)

def main():
    running = True

    while running:
```

```
SCREEN.fill(DARK_GRAY)

for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

    engine.handle_event(event)

if engine.should_exit:
    running = False

engine.handle_input()
engine.update()
engine.render(SCREEN)

pygame.display.flip()
clock.tick(FPS)

pygame.quit()

if __name__ == "__main__":
    main()
```

6. What happened to Task 2?

The Task 2 functionality is still there.

When the timer reaches zero:

</> Python

```
if self.time_left_frames <= 0:  
    self.game_over = True  
    return
```

The game stops updating the target because:

</> Python

```
if self.game_over:  
    return
```

Then `render()` shows the Game Over menu instead of the normal game.

So Task 2 hasn't been removed; **Task 3 extends the Game Over screen with the difficulty choices.**

7. Testing Task 3

Test Easy

Run:

</> Bash

```
python main.py
```

Wait until Game Over.

You should see:

</> Plain text

GAME OVER

Final Score: ...

Final Accuracy: ...

Difficulty: Medium

1 - Easy

2 - Medium

3 - Hard

4 - Exit

Press:

</> Plain text

1

A completely new round should start.

Verify:

- Score becomes 0
- Accuracy becomes 0.0%
- Hits become 0
- Misses become 0
- Timer starts at 30s

- A new target appears
 - Target is noticeably **larger**
 - Target stays alive longer
-

Test Medium

After the next round ends, press:

⌘ Plain text

2

You should get:

- Score = 0
- Accuracy = 0.0%
- New target
- 30-second timer
- Medium-sized target
- Original target behavior

Medium is essentially the original configuration:

⌘ Plain text

```
base_radius = 40
min_radius = 12
lifespan = 90 frames
```

Test Hard

Let the round end and press:

</> Plain text

3

The new target should be:

- Smaller
- Shrink to a smaller minimum
- Disappear much faster

The Hard configuration is:

</> Plain text

```
base_radius = 30  
min_radius = 8  
lifespan = 60 frames
```

So you should notice that you have significantly less time to hit each target.

8. Verify that the game state actually resets

This is important for your lab demonstration.

Before Game Over

Get something like:

⟨/⟩ Plain text

Score: 7

Accuracy: 70%

Then allow the timer to finish.

At Game Over you might see:

⟨/⟩ Plain text

Final Score: 7

Final Accuracy: 70.0%

Now press 1 .

The new round should **not** start with:

⟨/⟩ Plain text

Score: 7

It should start with:

⟨/⟩ Plain text

Score: 0

Accuracy: 0.0%

Time: 30s

This confirms that:

</> Python

```
self.hits = 0
self.misses = 0
self.score = 0
self.time_left_frames = self.round_seconds * 60
```

are being reset.

9. Verify repeated replay

This is another good test:

</> Plain text

```
Round 1
↓
Game Over
↓
Easy
↓
Round 2
↓
Game Over
↓
Hard
↓
Round 3
↓
Game Over
↓
Medium
```



Round 4

Each round should independently start with a clean score and timer.

Finally, from Game Over press:

</> Plain text

4

The game should close.

One important thing

I have **not added sound effects**, because that is Task 4.

I also haven't introduced classes, state machines, external configuration files, or other architecture that isn't necessary. The existing `GameEngine` remains responsible for the game state, and `Target` remains responsible for target behavior.

Your Task 1 collision fix also remains intact because the target's collision area continues to use its **current visual radius**, regardless of which difficulty is selected.

Add simple sound feedback for:

1. A successful target hit.
2. A normal miss when the player clicks outside the target.
3. A target timing out/expiring, which should also count as a miss.
4. The round ending when the 30-second timer reaches zero.

Please use Pygame's existing sound functionality and keep the implementation simple and suitable for a student Software Engineering lab.

If separate audio files are needed, please tell me exactly:

- Which files I need.
- What filenames to use.
- Where to place them in the project.
- What format they should be in.

Please avoid adding unnecessary libraries or complicated audio architecture.

Keep the existing Task 1, Task 2 and Task 3 functionality unchanged.

In particular:

- Don't break the shrinking target or collision detection.
- Don't remove the Game Over screen.
- Don't remove Easy/Medium/Hard replay.
- Don't change the scoring or accuracy logic unnecessarily.

Before giving me the code, explain briefly how you are going to connect each sound to the appropriate game event.

Then give me the complete updated code for the files that need to change.

Also give me clear instructions for adding any required sound files and explain how I can test all four sound events manually.

This is the final coding task, so please implement **Task 4 only** and don't add unrelated features.

Yes. For Task 4, the cleanest approach is to use **Pygame's built-in `pygame.mixer.Sound`** and add four small `.wav` files. No new Python library is needed.

I'll keep Tasks 1–3 unchanged: collision still uses the shrinking radius, Game Over remains, and Easy/Medium/Hard replay remains.

1. How each sound will be connected

There are four existing game events where sound can be added:

Event	Existing code location	Sound
Successful hit	<code>_handle_click()</code> when target contains the click	<code>hit.wav</code>
Normal miss	<code>_handle_click()</code> when click is outside target	<code>miss.wav</code>
Target expires	<code>update()</code> when <code>target.expired()</code>	<code>timeout.wav</code>
30-second round ends	<code>update()</code> when timer reaches zero	<code>game_over.wav</code>

So the logic remains essentially:

</> Plain text

```
Click target
  ↓
hit.wav

Click outside target
  ↓
miss.wav

Target reaches its lifespan
  ↓
```

timeout.wav

30-second timer reaches zero

↓

game_over.wav

↓

Game Over screen

The scoring and accuracy calculations remain exactly as they are.

2. Sound files you need

Create this folder:

</> Plain text

target-aim-trainer-main/

|

├─ main.py

├─ requirements.txt

├─ README.md

|

├─ game/

| └─ game_engine.py

| └─ target.py

|

└─ sounds/

├─ hit.wav

├─ miss.wav

├─ timeout.wav

└─ game_over.wav

Use these **exact filenames**:

</> Plain text

```
hit.wav  
miss.wav  
timeout.wav  
game_over.wav
```

Format

Use:

- **WAV**
- Short sound effects
- Mono or stereo is fine
- PCM WAV is safest
- Around 0.1–1 second per effect is enough

You can create/download simple sound effects, but don't add an audio package just to generate them. Pygame can play the WAV files directly.

3. Updated `game/game_engine.py`

Only this file needs actual game-logic changes.

`main.py` does **not** need to change for Task 4 because `pygame.init()` already initializes Pygame's modules, including the mixer in the normal setup. Your existing `main.py` can remain as it is from Task 3.

Updated `game/game_engine.py` — Task 4



```
import pygame
import random
from .target import Target

# Game Engine

WHITE = (255, 255, 255)
RED = (220, 60, 60)

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.margin = 60
        self.hud_height = 60

        self.round_seconds = 30
        self.difficulties = {
            "Easy": {
                "base_radius": 50,
                "min_radius": 18,
                "lifespan_frames": 120
            },
            "Medium": {
                "base_radius": 40,
                "min_radius": 12,
```

```
        "lifespan_frames": 90
    },
    "Hard": {
        "base_radius": 30,
        "min_radius": 8,
        "lifespan_frames": 60
    }
}
```

```
self.difficulty = "Medium"
```

```
self.hits = 0
```

```
self.misses = 0
```

```
self.score = 0
```

```
self.font = pygame.font.SysFont("Arial", 26)
```

```
self.game_over = False
```

```
self.should_exit = False
```

```
self.hit_sound = pygame.mixer.Sound("sounds/hit.wav")
```

```
self.miss_sound = pygame.mixer.Sound("sounds/miss.wav")
```

```
self.timeout_sound = pygame.mixer.Sound("sounds/timeout.wav")
```

```
self.game_over_sound = pygame.mixer.Sound("sounds/game_over.wav")
```

```
self._start_new_game()
```

```
def _spawn_target(self):
```

```
    x = random.randint(self.margin, self.width - self.margin)
```

```
y = random.randint(
    self.margin + self.hud_height,
    self.height - self.margin
)

settings = self.difficulties[self.difficulty]

return Target(
    x,
    y,
    settings["base_radius"],
    settings["min_radius"],
    settings["lifespan_frames"]
)

def _start_new_game(self):
    self.hits = 0
    self.misses = 0
    self.score = 0
    self.time_left_frames = self.round_seconds * 60
    self.game_over = False
    self.target = self._spawn_target()

def handle_event(self, event):
    if self.game_over:
        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_1:
                self.difficulty = "Easy"
```

```
        self._start_new_game()

    elif event.key == pygame.K_2:
        self.difficulty = "Medium"
        self._start_new_game()

    elif event.key == pygame.K_3:
        self.difficulty = "Hard"
        self._start_new_game()

    elif event.key == pygame.K_4:
        self.should_exit = True

    return

if event.type == pygame.MOUSEBUTTONDOWN:
    self._handle_click(event.pos)

def _handle_click(self, pos):
    x, y = pos

    if self.target.contains_point(x, y):
        self.hits += 1
        self.score += 1
        self.hit_sound.play()
        self.target = self._spawn_target()
    else:
        self.misses += 1
```

```
        self.miss_sound.play()

def handle_input(self):
    # Reserved for continuously-held-key input; this game is
    # entirely mouse-driven, so there's nothing to poll here.
    pass

def update(self):
    if self.game_over:
        return

    self.time_left_frames -= 1

    if self.time_left_frames <= 0:
        self.game_over = True
        self.game_over_sound.play()
        return

    self.target.update()

    if self.target.expired():
        self.misses += 1
        self.timeout_sound.play()
        self.target = self._spawn_target()

def accuracy(self):
    total = self.hits + self.misses
```

```
    if total == 0:
        return 0.0

    return round(100 * self.hits / total, 1)

def render(self, screen):
    if self.game_over:
        game_over_text = self.font.render(
            "GAME OVER",
            True,
            WHITE
        )

        score_text = self.font.render(
            f"Final Score: {self.score}",
            True,
            WHITE
        )

        accuracy_text = self.font.render(
            f"Final Accuracy: {self.accuracy()}%",
            True,
            WHITE
        )

        difficulty_text = self.font.render(
            f"Difficulty: {self.difficulty}",
            True,
```

```
        WHITE
    )

    easy_text = self.font.render(
        "1 - Easy",
        True,
        WHITE
    )

    medium_text = self.font.render(
        "2 - Medium",
        True,
        WHITE
    )

    hard_text = self.font.render(
        "3 - Hard",
        True,
        WHITE
    )

    exit_text = self.font.render(
        "4 - Exit",
        True,
        WHITE
    )

    screen.blit(
```

```
        game_over_text,
        (
            self.width // 2 - game_over_text.get_width() // 2,
            60
        )
    )

    screen.blit(
        score_text,
        (
            self.width // 2 - score_text.get_width() // 2,
            110
        )
    )

    screen.blit(
        accuracy_text,
        (
            self.width // 2 - accuracy_text.get_width() // 2,
            150
        )
    )

    screen.blit(
        difficulty_text,
        (
            self.width // 2 - difficulty_text.get_width() // 2,
            190
        )
    )
```

```
    )  
)  
  
screen.blit(  
    easy_text,  
    (  
        self.width // 2 - easy_text.get_width() // 2,  
        250  
    )  
)  
  
screen.blit(  
    medium_text,  
    (  
        self.width // 2 - medium_text.get_width() // 2,  
        290  
    )  
)  
  
screen.blit(  
    hard_text,  
    (  
        self.width // 2 - hard_text.get_width() // 2,  
        330  
    )  
)  
  
screen.blit(  

```

```
        exit_text,  
        (  
            self.width // 2 - exit_text.get_width() // 2,  
            370  
        )  
    )  
  
    return  
  
    r = int(self.target.visual_radius())  
  
    pygame.draw.circle(  
        screen,  
        RED,  
        (self.target.x, self.target.y),  
        r  
    )  
  
    pygame.draw.circle(  
        screen,  
        WHITE,  
        (self.target.x, self.target.y),  
        r,  
        2  
    )  
  
    score_text = self.font.render(  
        f"Score: {self.score}",
```

```
        True,  
        WHITE  
    )  
  
    screen.blit(score_text, (10, 10))  
  
    seconds_left = max(  
        0,  
        self.time_left_frames // 60  
    )  
  
    timer_text = self.font.render(  
        f"Time: {seconds_left}s",  
        True,  
        WHITE  
    )  
  
    screen.blit(  
        timer_text,  
        (self.width - 140, 10)  
    )  
  
    acc_text = self.font.render(  
        f"Accuracy: {self.accuracy()}%",  
        True,  
        WHITE  
    )
```

```
screen.blit(  
    acc_text,  
    (self.width // 2 - 90, 10)  
)
```

4. What exactly changed?

There are only a few additions.

Sound loading

In `__init__()`:

</> Python

```
self.hit_sound = pygame.mixer.Sound("sounds/hit.wav")  
self.miss_sound = pygame.mixer.Sound("sounds/miss.wav")  
self.timeout_sound = pygame.mixer.Sound("sounds/timeout.wav")  
self.game_over_sound = pygame.mixer.Sound("sounds/game_over.wav")
```

This loads the four sound effects once when the game starts.

Successful hit

The existing scoring logic stays unchanged:

</> Python

```
self.hits += 1  
self.score += 1
```

We simply add:

</> Python

```
self.hit_sound.play()
```

So:

</> Plain text

```
Hit  
↓  
hits += 1  
score += 1  
sound  
↓  
new target
```

Normal miss

Existing:

</> Python

```
self.misses += 1
```

Now:

</> Python

```
self.misses += 1
```

```
self.miss_sound.play()
```

So the accuracy calculation isn't changed.

Target timeout

Existing behavior:

</> Python

```
if self.target.expired():
    self.misses += 1
    self.target = self._spawn_target()
```

Now:

</> Python

```
if self.target.expired():
    self.misses += 1
    self.timeout_sound.play()
    self.target = self._spawn_target()
```

So a timeout still counts as a miss exactly as before.

Round ending

Existing:

</> Python

```
if self.time_left_frames <= 0:
    self.game_over = True
```

```
return
```

Now:

⌕ Python

```
if self.time_left_frames <= 0:
    self.game_over = True
    self.game_over_sound.play()
    return
```

The Game Over screen still appears because `render()` already checks `self.game_over` .

5. Task 1, 2 and 3 remain intact

Task 1 — Collision

Still uses:

⌕ Python

```
self.target.contains_point(x, y)
```

and your fixed `target.py` should still contain:

⌕ Python

```
return math.hypot(self.x - x, self.y - y) <= self.visual_radius()
```

So the collision area continues shrinking with the visible `target`. `</>` `target`

Task 2 — Game Over

The timer still does:

`</>` Python

```
self.game_over = True
```

and the Game Over screen remains in `render()` .

The README specifically describes Task 4 as adding sound for successful hits, misses, timed-out targets, and the round ending, which matches these four event points. [📖 README](#)

Task 3 — Difficulty

The existing Easy/Medium/Hard configuration remains unchanged:

`</>` Python

```
"Easy"  
"Medium"  
"Hard"
```

and selecting `1` , `2` , or `3` still starts a fresh 30-second round.

6. Add the sound files

Your project should ultimately look like:

</> Plain text

```
target-aim-trainer-main/  
|  
├─ main.py  
├─ requirements.txt  
├─ README.md  
|  
├─ game/  
|   ├─ game_engine.py  
|   └─ target.py  
|  
└─ sounds/  
    ├─ hit.wav  
    ├─ miss.wav  
    ├─ timeout.wav  
    └─ game_over.wav
```

Important

Run the game from the **project root**, the same place where `main.py` is located:

</> Bash

```
python main.py
```

The code uses:

</> Python

```
"sounds/hit.wav"
```

rather than an absolute Windows path, so the project remains portable.

7. Manual testing

You can test all four sounds separately.

Test 1 — Successful hit

Start:

</> Bash

```
python main.py
```

Click directly inside the target.

Expected:

</> Plain text

```
Target hit  
→ score increases  
→ hit sound plays  
→ new target appears
```

Test 2 — Normal miss

Click somewhere clearly outside the visible target.

Expected:

</> Plain text

```
miss sound plays  
→ Miss count increases  
→ target remains
```

The score should **not** increase.

Test 3 — Target timeout

Don't click the target.

Wait for it to shrink completely.

Expected:

</> Plain text

```
Target expires  
→ timeout sound plays  
→ miss count increases  
→ new target appears
```

This is separate from a normal mouse-click miss, so you can demonstrate that the two miss events have different sounds.

Test 4 — Round ending

Let the 30-second timer reach zero.

Expected:

</> Plain text

30 seconds

↓

game_over sound

↓

GAME OVER screen

↓

Final Score

Final Accuracy

Easy / Medium / Hard / Exit

The target should stop moving/updating.

Test 5 — Make sure Task 3 still works

At Game Over press:

</> Plain text

1

A new Easy round should start.

Then allow it to finish and try:

</> Plain text

2

and later:

</> Plain text

3

Each should start a new 30-second round with reset score/accuracy and the appropriate target behavior.

Finally:

</> Plain text

4

should exit.

One thing to watch for

If you get an error like:

</> Plain text

```
pygame.error: mixer not initialized
```

or:

</> Plain text

```
FileNotFoundError
```

check that:

1. You are running `python main.py` from the project root.
2. The folder is exactly named `sounds`.
3. The filenames are exactly:
 - `hit.wav`
 - `miss.wav`
 - `timeout.wav`
 - `game_over.wav`

No changes to `requirements.txt` are necessary because **Pygame already provides the sound functionality**.

This completes **Task 4 only** and doesn't add any unrelated functionality.